Lecture 21 Practice Questions and Solutions

Question 1 (q1_sol.py)

Sold Out!

Orders for the campus coffee shop are kept in a list of strings, oldest order first. For example:

```
orders = ['latte', 'muffin', 'oat latte', 'tea', 'oat latte',
          'scone', 'oat latte']
```

Sometimes the shop runs out of an item, and so those items need to be removed from the order list. The barista also needs to know how many orders were removed.

Write a function called remove_sold_out(orders, bad_item) that:

- removes *every* copy of bad_item from the list orders, using the list's remove method
- returns (not prints!) the number of items removed

The function should change the orders list itself in-place, i.e. it should NOT make a new list.

Your remove_sold_out(orders, bad_item) function should work with code like this:

```
orders = ['latte', 'muffin', 'oat latte', 'tea', 'oat latte',
          'scone', 'oat latte']
n = remove_sold_out(orders, 'oat latte')
print(n)        # prints 3
print(orders)   # prints ['latte', 'muffin', 'tea', 'scone']

# orders is now: ['latte', 'muffin', 'tea', 'scone']
n = remove_sold_out(orders, 'hot chocolate')
print(n)        # prints 0
print(orders)   # prints ['latte', 'muffin', 'tea', 'scone']
```

Sample Solution

```
def remove_sold_out(orders, bad_item):
    count = 0
    while bad_item in orders:
        orders.remove(bad_item)
        count += 1
    return count

orders = ['latte', 'muffin', 'oat latte', 'tea', 'oat latte',
          'scone', 'oat latte']
n = remove_sold_out(orders, 'oat latte')
print(n)        # prints 3
print(orders)   # prints ['latte', 'muffin', 'tea', 'scone']
```

```
# orders is now: ['latte', 'muffin', 'tea', 'scone']
n = remove_sold_out(orders, 'hot chocolate')
print(n)        # prints 0
print(orders)   # prints ['latte', 'muffin', 'tea', 'scone']
```

Question 2 (q2_sol.py)

SFU gives semesters a code number as follows:

```
Semester   Code
---------------
 Spring      1
 Summer      4
 Fall        7
```

Write a program that stores the semesters and their codes in a dictionary. The semesters are the keys, and the codes are the values.

Then write a program that repeatedly asks the user to enter a semester, and, using the dictionary, prints the corresponding code. It should stop when they press the <return> key. If the user enters an unknown semester, then print a helpful error message.

Important: the *case* of the semester name does not matter. So, for example, "spring", "Spring", and "SPRING" should all be treated the same. The output the semester name is always capitalized, e.g. "Summer", "Spring, "Fall".

Sample Run

```
Enter semester: Spring
Spring code: 1

Enter semester: SUMMER
Summer code: 4

Enter semester: Winter
Unknown semester

Enter semester: <return>
All done!
```

Sample Solution

```
codes = {'spring': 1, 'summer': 4, 'fall': 7}

while True:
    semester = input('Enter semester: ')
    semester = semester.strip().lower()
    if semester == '':
        break
    elif semester in codes:
        code = codes[semester]
        print(f'{semester.capitalize()} code: {code}')
    else:
        print('Unknown semester')
```

```
    print()

print('All done!')
```

Question 3 (q3_sol.py)

SFU term codes are 4-digit numbers, where each number has a specific meaning. For example, the term code 1267 means Fall 2026 because:

- the first digit is a 1 (for this century)
- the next two digits are 26, the last two digits of the year 2026
- the last digit is a 7, which is the semester code for Fall

Using your semesters dictionary, write code that takes a semester name and a year and prints the term code. It should read in the semester name, the year, and then print the term code. You do *not* need to repeatedly ask the user, i.e. it can stop after the printing the term code.

3 Sample Runs

```
Enter semester: Fall
Enter year: 2026
Fall 2026 term code: 1267

Enter semester: Spring
Enter year: 2027
Spring 2027 term code: 1271

Enter semester: Summer
Enter year: 2026
Summer 2026 term code: 1264
```

Use this dictionary in your program:

```
codes = {'spring': 1, 'summer': 4, 'fall': 7}
```

Sample Solution

```
codes = {'spring': 1, 'summer': 4, 'fall': 7}

# get the input
semester = input('Enter semester: ').strip().lower()
year = input('Enter year: ').strip()

# process the input: construct the term code
term_code = '1' + year[-2:] + str(codes[semester])

# print the result
print(f'{semester.capitalize()} {year} term code: {term_code}')
```

Question 4 (q4_sol.py)

Write a program that repeatedly asks the user to enter a term *code number* and the prints the corresponding semester name. It should stop when the user types <return>, and print a helpful message if an invalid code is typed. Use a dictionary to efficiently store the semester codes.

Here are the semester codes:

```
Semester   Code
---------------
 Spring      1
 Summer      4
 Fall        7
```

Sample Run

```
Enter semester code: 4
4 is Summer

Enter semester code: 8
8 is invalid

Enter semester code: 1
1 is Spring

Enter semester code: <return>
All done!
```

Sample Solution

```
codes = {1: 'Spring', 4: 'Summer', 7: 'Fall'}

while True:
    code = input('Enter semester code: ')
    if code == '':
      break
    code = int(code.strip())
    if code in codes:
        semester = codes[code]
        print(f'{code} is {semester}')
    else:
        print(f'{code} is invalid')
    print()

print('All done!')
```